

DEAKIN UNIVERSITY

FULL STACK DEVELOPMENT: SECURE FRONTEND APPLICATIONS

ONTRACK SUBMISSION

Home Page

Submitted By:

Romil BIJARNIA

s222528574

2026/08/10 22:36

Tutor:

Nisha KUMARI

Task Outcomes	Supports
TLO1 Demonstrate engagement with the following unit learning outcomes (legacy)	ULO Unit Learning Outcome 1

August 10, 2026



Task P3

Home Page

Evidence report for the DEV@Deakin home page migrated to React with Vite and implemented with reusable components, data-driven content, media, continuation controls, and a complete responsive footer.

STUDENT

Romil Bijarnia

TASK

P3 - Home Page

APPLICATION

DEV@Deakin

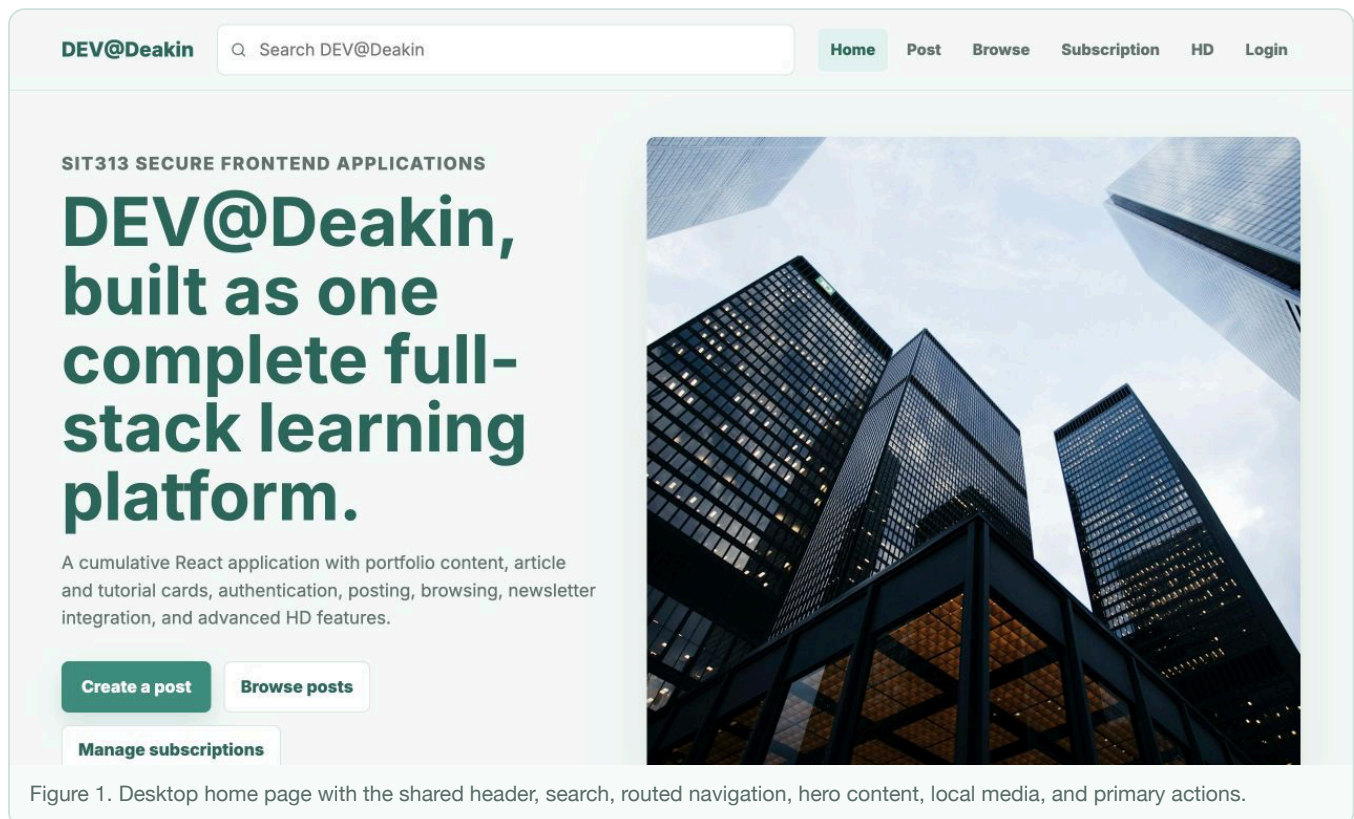


Figure 1. Desktop home page with the shared header, search, routed navigation, hero content, local media, and primary actions.

Implementation outcome

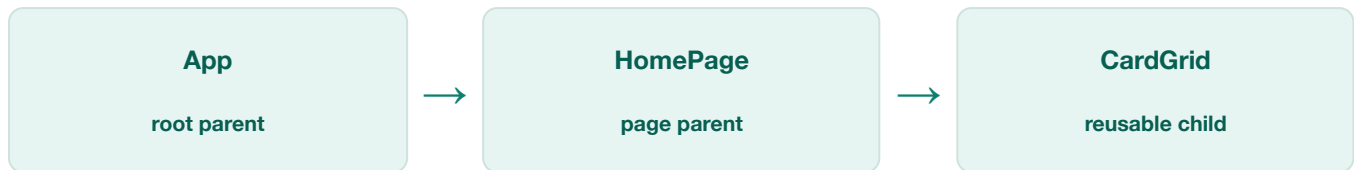
The existing frontend is now a React 18 single-page application served and bundled by Vite. The home route composes focused sections rather than placing the interface in one oversized component.

Design outcome

The page includes three featured articles, three featured tutorials, both required "See all" controls, a newsletter panel, local images and Lucide icons, and the full footer structure.

1 React component architecture

App is the application shell. It composes the shared Header, the route selected by React Router, and the shared Footer. The home route renders HomePage, which composes the hero, profile band, two featured sections, Newsletter, and photo gallery. Repeated cards are delegated to CardGrid, with content supplied through props.



Component	Responsibility	Data or behaviour
App	Shared application shell and routes	Maintains current user state and passes logout behaviour to Header
Header	Logo, search, navigation, login/sign-out control	Receives user and onLogout from its parent
HomePage	Composes every home-page section	Passes the article and tutorial arrays to CardGrid
CardGrid	Renders consistent content cards	Receives items and kind props and maps each record
Newsletter	Controlled email form	Owns email, loading, and result state with React hooks
Footer	Shared navigation, support, social, and legal areas	Uses semantic links and labelled navigation groups

```

function CardGrid({ items, kind }) {
  return (
    <div className="media-grid">
      {items.map((item) => (
        <article className="media-card" key={item.id}>
          <img src={item.image} alt={` ${item.title} ${kind}`} />
          { /* title, description, rating and author */ }
        </article>
      ))}
    </div>
  );
}

```

Why this meets the parent-children requirement

The hierarchy gives each component one clear responsibility. Data moves from HomePage to CardGrid through props, while the newsletter keeps its form state locally because no other section needs it.

2 React and Vite migration

frontend/index.html provides the root element, and src/main.jsx mounts App with createRoot, React.StrictMode, and BrowserRouter. Styles are imported from src/styles.css. The P3 home page can be reviewed without Firebase configuration; npm run frontend starts the local Vite server.

3 Arrays and `map()`

`frontend/src/content.js` exports separate `articles` and `tutorials` arrays. Every item has a stable `id`, `title`, `description`, `author`, `rating`, and `image path`. `HomePage` passes each array to the same `CardGrid` child, and `items.map()` transforms the records into cards. The stable identifier is used as the React key.

```
export const articles = [
  {
    id: 'article-react-security',
    title: 'Designing Secure React Interfaces',
    description: 'Practical patterns for forms, validation, routing, and safe user feedback.',
    author: 'Romil Bijarnia',
    rating: '5.0',
    image: '/assets/project-portfolio.jpg'
  },
  // two further records with the same shape
];

<CardGrid items={articles} kind="article" />
<CardGrid items={tutorials} kind="tutorial" />
```

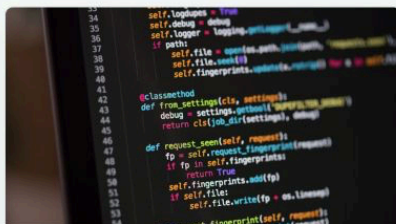


Here is what I have done so far

I have built the DEV@Deakin platform from a static portfolio into a connected app with route-based pages, reusable React components, API communication, local data persistence, form validation, and secure-authentication concepts.

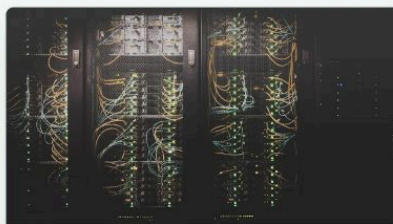
FEATURED ARTICLES

Articles rendered from an array with `map`



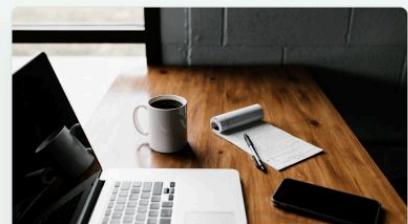
Designing Secure React Interfaces

Practical patterns for forms, validation,



Connecting APIs to Frontend Pages

How frontend forms send requests, handle



Building a Newsletter Signup

A small full-stack feature that turns a static

Figure 2. Profile content and three Featured Articles rendered from the article array. Each card presents local media, a title, description, rating, and author.

Required continuation controls

See all articles

is a semantic React Router Link to `/browse`.

See all tutorials

links to `/hd#tutorial-library`. Both controls appear below their corresponding three-card section.

Media and icons

Hero, card, and gallery photographs are local assets under `frontend/public/assets`. Search, star, send, and social icons come from `lucide-react`; decorative icons are hidden from assistive technology.

4 Complete footer and responsive layout

The newsletter panel presents the required daily-insider heading, email field, and Subscribe control. The footer below it is a reusable component with four desktop columns: the DEV@Deakin brand statement, Explore navigation, Support links, and Deakin social links. A separate legal row contains Privacy Policy, Terms, and Code of Conduct.

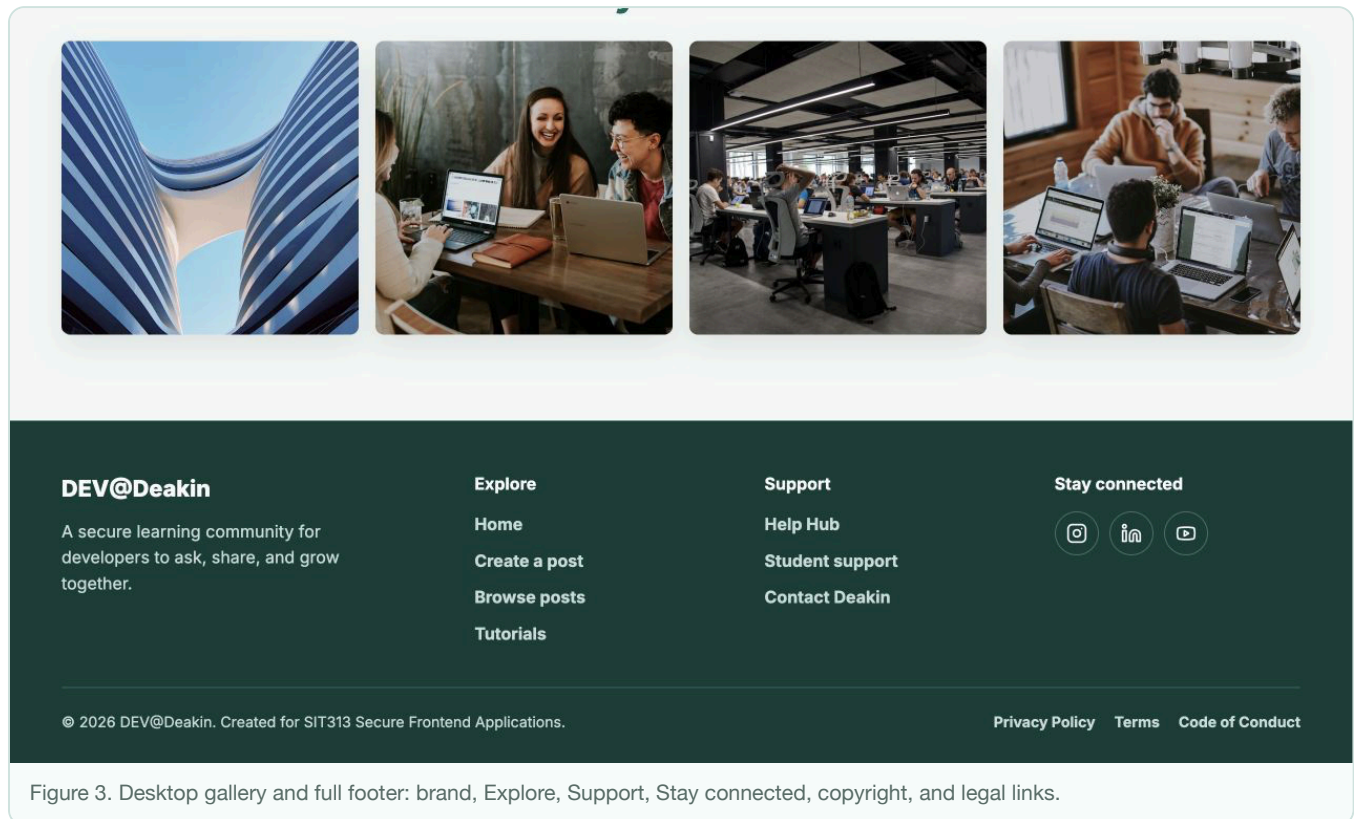


Figure 3. Desktop gallery and full footer: brand, Explore, Support, Stay connected, copyright, and legal links.

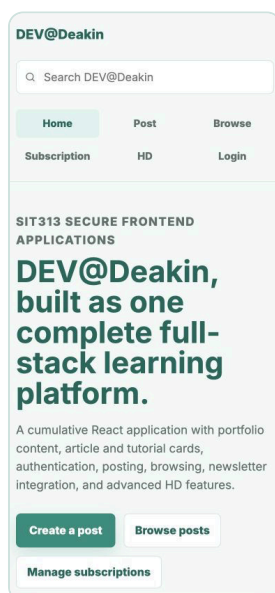


Figure 4. Mobile header and hero at 390 × 844.

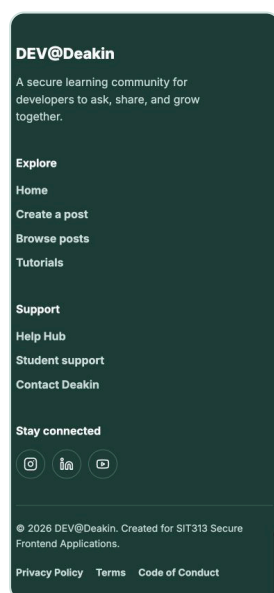


Figure 5. Mobile footer at 390 × 844.

Responsive rules

At widths below 940 px, the header, hero, newsletter, and footer reflow and the card grid changes from three columns to two. Below 640 px, content cards and footer columns stack into one column.

The navigation wraps without horizontal overflow, action buttons remain touch-sized, the newsletter form becomes vertical, and the legal row stacks. Fluid headings use `clamp()`, while local images retain their intended crop with `object-fit`.

Accessibility details

Landmark elements and labelled navigation groups give the page semantic structure. Images have descriptive alternative text, focus styles remain visible, and the reduced-motion media query removes transitions for users who request it.

5 Requirement and build check

The task snapshot was checked against the P3 brief and then verified from an isolated dependency installation so generated dependencies were not added to the Desktop submission source.

✓

React migration

The frontend mounts through React 18 and is served and bundled with Vite.

✓

Parent-children structure

App, HomePage, CardGrid, Newsletter, Header, and Footer have separated responsibilities.

✓

Array-driven articles

Three article objects are stored in `content.js` and rendered with `map()` and stable keys.

✓

Required design sections

Featured Articles, Featured Tutorials, both "See all" controls, the newsletter panel, and the complete footer are present.

✓

Media and icons

Local photographs and named Lucide icon imports are used throughout the page.

✓

Repository hygiene

The root `.gitignore` excludes `node_modules/`, build output, environment files, Firebase local state, and runtime data.

PRODUCTION BUILD

Passed

MODULES TRANSFORMED

1,597

WALKTHROUGH DURATION

2:29

Build result

`npm run build` completed successfully with Vite 6.4.3 after 1,597 modules were transformed. Vite emitted a non-blocking large-chunk advisory because the cumulative application includes Firebase; there were no build errors.

Local evidence package

Task source Internal GitLab: Task P3 README and run instructions React component implementation	Walkthrough Panopto walkthrough (2:29) Local MP4 copy Covers the interface, component hierarchy, mapped data, responsive behaviour, and build result.
---	---

Run from the source root with `npm ci` followed by `npm run frontend`. Open `http://127.0.0.1:5173`. Firebase configuration is not required to review the P3 home page.